GHANA COMMUNICATION TECHNOLOGY UNIVERSITY

JAVA PROGRAMING

BY

ELVIS NII BORTEY 1724063167

LEVEL 300

BIT TOP UP (EVENING)

DATE: 13TH JANUARY 2025

Answer all questions (60 marks)

Q1:

You are required to write a Java program that calculates the standard deviation, mean, and median of a given set of numbers. The program should take input from the user for the array of numbers and display the calculated statistics.

Your program should perform the following tasks:

i. Prompt the user to enter the number of elements in the array.

4 Marks

ii. Accept the user's input for each element of the array.

5 Marks

iii. Assign the functions used to calculate mean, median and standard deviation to variables mean, median and standard deviation

3 Marks

iv. Display the mean,median and standarddeviation

3 Marks

v. Calculate the mean (average) of the entered numbers.

4 Marks

vi. Calculate the median (middle value) of the entered numbers.

5 Marks

vii. Calculate the standard deviation of the entered numbers.

6 Marks

Please follow these guidelines while writing your program:

- Use appropriate data types and variables to store the user input and calculated statistics.
- Implement separate methods to calculate the mean, median, and standard deviation.
- Handle cases where the number of elements is less than 2 for the standard deviation calculation, and provide a suitable error message in such cases.
- If the number of elements is even for calculating the median, you should take the average of the two middle values.
- Sub questions i-iv should be implemented in the main function

Ensure that your program is user-friendly, and it provides meaningful messages to guide the user through the input process. Also, consider adding comments to explain the logic and purpose of each part of your code.

Answer

```
import java.util.Arrays;
import java.util.Scanner;
public class StatisticsCalculator {
    // Method to calculate the mean
    public static double calculateMean(double[] numbers) {
        double sum = 0;
        for (double num : numbers) {
            sum += num;
        }
        return sum / numbers.length;
    }
    // Method to calculate the median
    public static double calculateMedian(double[] numbers) {
        Arrays.sort(numbers);
        int n = numbers.length;
        if (n % 2 == 0) {
            return (numbers[n / 2 - 1] + numbers[n / 2]) / 2;
        } else {
            return numbers[n / 2];
        }
    }
```

```
    // Method to calculate the standard deviation
    public static double calculateStandardDeviation(double[] numbers) {
        if (numbers.length < 2) {
            throw new IllegalArgumentException("Standard deviation requires at least two elements.");
        }
        double mean = calculateMean(numbers);
        double sumOfSquares = 0;
        for (double num : numbers) {
            sumOfSquares += Math.pow(num - mean, 2);
        }
        return Math.sqrt(sumOfSquares / (numbers.length - 1));
    }
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        // Prompt the user to enter the number of elements in the array
        System.out.print("Enter the number of elements: ");
        int n = scanner.nextInt();
        // Validate the number of elements
        if (n <= 0) {
            System.out.println("The number of elements must be greater than zero.");
            return;
        }
```

```
        // Accept the user's input for each element of the array
        double[] numbers = new double[n];
        System.out.println("Enter the elements:");
        for (int i = 0; i < n; i++) {
            System.out.print("Element " + (i + 1) + ": ");
            numbers[i] = scanner.nextDouble();
        }
        // Assign functions to variables and display the results
        try {
            double mean = calculateMean(numbers);
            double median = calculateMedian(numbers);
            double standardDeviation = calculateStandardDeviation(numbers);
            System.out.println("\nResults:");
            System.out.printf("Mean: %.2f\n", mean);
            System.out.printf("Median: %.2f\n", median);
            System.out.printf("Standard Deviation: %.2f\n", standardDeviation);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
        scanner.close();
    }
}
```

Q2

A. Create a class name **Emolument,** to represent an emolument. The class contains the following:

i. Two double data fields name **basic_salary** and **tax_relief** that represent basic salary and tax relief amounts. This data fields should be encapsulated.

2 Marks

ii. A constructor that creates emolument with the specified Basic Salary and Tax Relief.

3 Marks

iii. A method named **getBasicsalary()** that returns the basic_salary.

1 Mark

iv. A method named **getTaxRelief()** that returns the tax_relief.

1 Mark

v. A method named **SSNIT()** that returns the computed SSNIT contribution of the emolument. Note: SSNIT contribution is 3.5% of Basic Salary.

2 Marks

vi. A method named **taxableIncome()** that returns the computed Taxable Income of the emolument. Note: Taxable Income = Basic salary – (Tax relief + SSNIT contribution).

2 Marks

Answer

```
public class Emolument {
    // Encapsulated data fields
    private double basic_salary;
    private double tax_relief;

    // Constructor
    public Emolument(double basic_salary, double tax_relief) {
        this.basic_salary = basic_salary;
        this.tax_relief = tax_relief;
    }

    // Getter method for basic salary
    public double getBasicSalary() {
        return basic_salary;
    }

    // Getter method for tax relief
    public double getTaxRelief() {
        return tax_relief;
```

```
    }

    // Method to compute SSNIT contribution (3.5% of Basic Salary)
    public double SSNIT() {
        return 0.035 * basic_salary;
    }

    // Method to compute taxable income
    public double taxableIncome() {
        return basic_salary - (tax_relief + SSNIT());
    }
}
```

B. Create a class named **MyEmolument** that inherits from Emolument class. The class contains the following:

i. Two double data fields name **basic_salary** and **tax_relief**, that represent basic salary and tax relief amounts. This data fields should be encapsulated.

2 Marks

ii. A **non-arg constructor** that creates a default emolument. The default values are **0** for basic salary and tax relief.

2 Marks

iii. A constructor that creates **MyEmolument** with the specified basic salary and Tax relief.

1 Mark

iv. A method named **incomeTax()** that return the computed Income Tax. Income Tax is calculated as follows:

a. The first 500.00 of Taxable Income, the tax rate is 5%.
b. The next 500.00 Taxable Income, the tax rate is 12.5%.
c. The rest, the tax rate is 17.5%.

3 Marks

v. A method named **totalDeduction()** that return the value of Total Deduction. Note: *Total Deduction = SSNIT Contribution + Income Tax*.

1 Mark

vi. A method named **netSalary()** that return the value of Net Salary.

Note: *Net Salary = Basic salary – Total Deduction.*

1 Mark

```
// Parent class Emolument (for inheritance purposes)
class Emolument {
    // Parent class can be empty or include shared functionality
}

// MyEmolument class that extends Emolument
public class MyEmolument extends Emolument {
    // Encapsulated data fields for basic salary and tax relief
    private double basic_salary;
    private double tax_relief;

    // Non-argument constructor with default values
    public MyEmolument() {
        this.basic_salary = 0.0;
        this.tax_relief = 0.0;
    }

    // Constructor with specified basic salary and tax relief
    public MyEmolument(double basic_salary, double tax_relief) {
        this.basic_salary = basic_salary;
```

```
    this.tax_relief = tax_relief;
}

// Getter and Setter for basic_salary
public double getBasicSalary() {
    return basic_salary;
}

public void setBasicSalary(double basic_salary) {
    this.basic_salary = basic_salary;
}

// Getter and Setter for tax_relief
public double getTaxRelief() {
    return tax_relief;
}

public void setTaxRelief(double tax_relief) {
    this.tax_relief = tax_relief;
}

// Method to calculate income tax
```

```
    public double incomeTax() {
        double taxableIncome = basic_salary - tax_relief;
        double incomeTax = 0.0;

        if (taxableIncome <= 500.0) {
            incomeTax = taxableIncome * 0.05;
        } else if (taxableIncome <= 1000.0) {
            incomeTax = 500 * 0.05 + (taxableIncome - 500) * 0.125;
        } else {
            incomeTax = 500 * 0.05 + 500 * 0.125 + (taxableIncome - 1000) * 0.175;
        }

        return incomeTax;
    }

    // Method to calculate total deduction (SSNIT Contribution + Income Tax)
    public double totalDeduction() {
        double ssnitContribution = basic_salary * 0.05; // Assume SSNIT Contribution is 5% of
basic salary
        double incomeTax = incomeTax();
        return ssnitContribution + incomeTax;
    }
```

```
    // Method to calculate net salary
    public double netSalary() {
        return basic_salary - totalDeduction();
    }

    // Main method to test the class
    public static void main(String[] args) {
        // Example usage
        MyEmolument emolument = new MyEmolument(2000.0, 150.0); // Specify basic salary and tax relief
        System.out.println("Basic Salary: " + emolument.getBasicSalary());
        System.out.println("Tax Relief: " + emolument.getTaxRelief());
        System.out.println("Income Tax: " + emolument.incomeTax());
        System.out.println("Total Deduction: " + emolument.totalDeduction());
        System.out.println("Net Salary: " + emolument.netSalary());
    }
}
```

C. Write a test program that creates MyEmolument object named **Staff_Salary** with below requirement.

i. Accept input from the user using the input dialog.

2 Marks

ii. Display the Basic Salary, Tax Relief, SSNIT Contribution, Taxable Income, Income Tax, Total Deduction and Net Salary as shown in the figure below.

7 Marks

Answer

```
import javax.swing.JOptionPane;


class MyEmolument {

   private double basicSalary, taxRelief, ssnitContribution, taxableIncome, incomeTax, totalDeduction, netSalary;


   public MyEmolument(double basicSalary) {

      this.basicSalary = basicSalary;

      calculateEmoluments();

   }


   private void calculateEmoluments() {

      taxRelief = basicSalary * 0.1;                // 10% Tax Relief

      ssnitContribution = basicSalary * 0.055;              // 5.5% SSNIT

      taxableIncome = basicSalary - (taxRelief + ssnitContribution);

      incomeTax = taxableIncome * 0.15;         // 15% Income Tax

      totalDeduction = ssnitContribution + incomeTax;

      netSalary = basicSalary - totalDeduction;
```

```
    }

    public void displayDetails() {
        String details = String.format(
            "Basic Salary: %.2f\nTax Relief: %.2f\nSSNIT Contribution: %.2f\nTaxable Income: %.2f\nIncome Tax: %.2f\nTotal Deduction: %.2f\nNet Salary: %.2f",
            basicSalary, taxRelief, ssnitContribution, taxableIncome, incomeTax, totalDeduction, netSalary);
        JOptionPane.showMessageDialog(null, details, "Staff Salary Details", JOptionPane.INFORMATION_MESSAGE);
    }
}

public class TestProgram {
    public static void main(String[] args) {
        String input = JOptionPane.showInputDialog("Enter Basic Salary:");
        double basicSalary = Double.parseDouble(input);

        MyEmolument staffSalary = new MyEmolument(basicSalary);
        staffSalary.displayDetails();
    }
}
```